

# Natural Language Processing

Dr. Oybek Eraliev,

---

Department of Computer Engineering

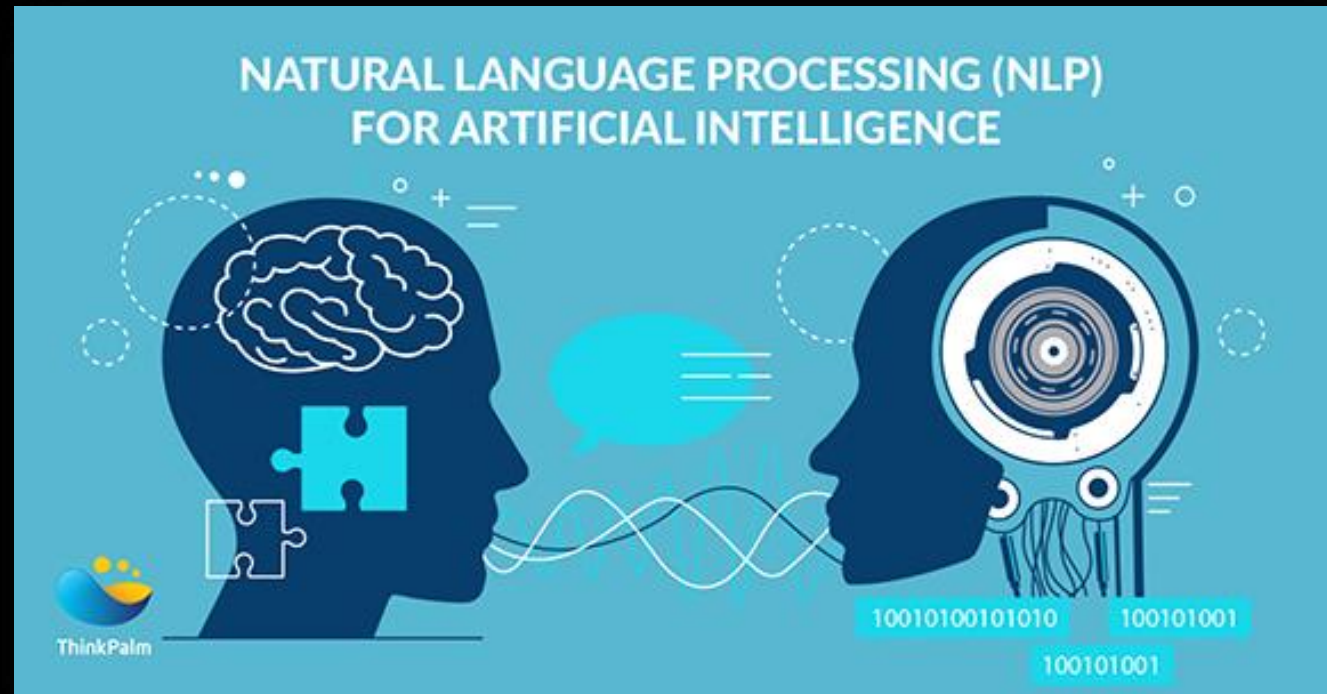
Inha University In Tashkent.

Email: [oybekeraliev7@gmail.com](mailto:oybekeraliev7@gmail.com)



# What is NLP?

- **Natural language processing (NLP)** is a subfield of computer science and artificial intelligence (AI) that uses machine learning to enable computers to understand and communicate with human language.



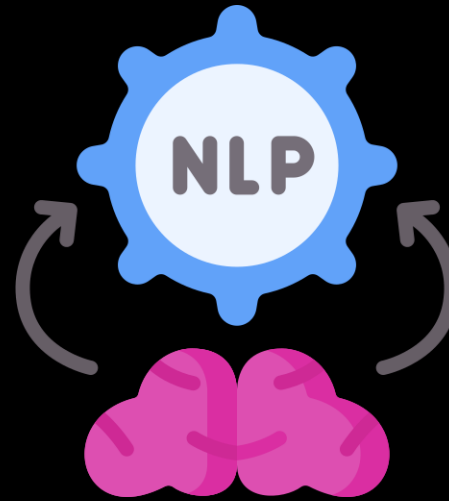
# NLP Tasks

Named entity  
recognition (NER)

Part-of-speech  
tagging (POS)

Speech recognition

Sentiment analysis



Language  
translation

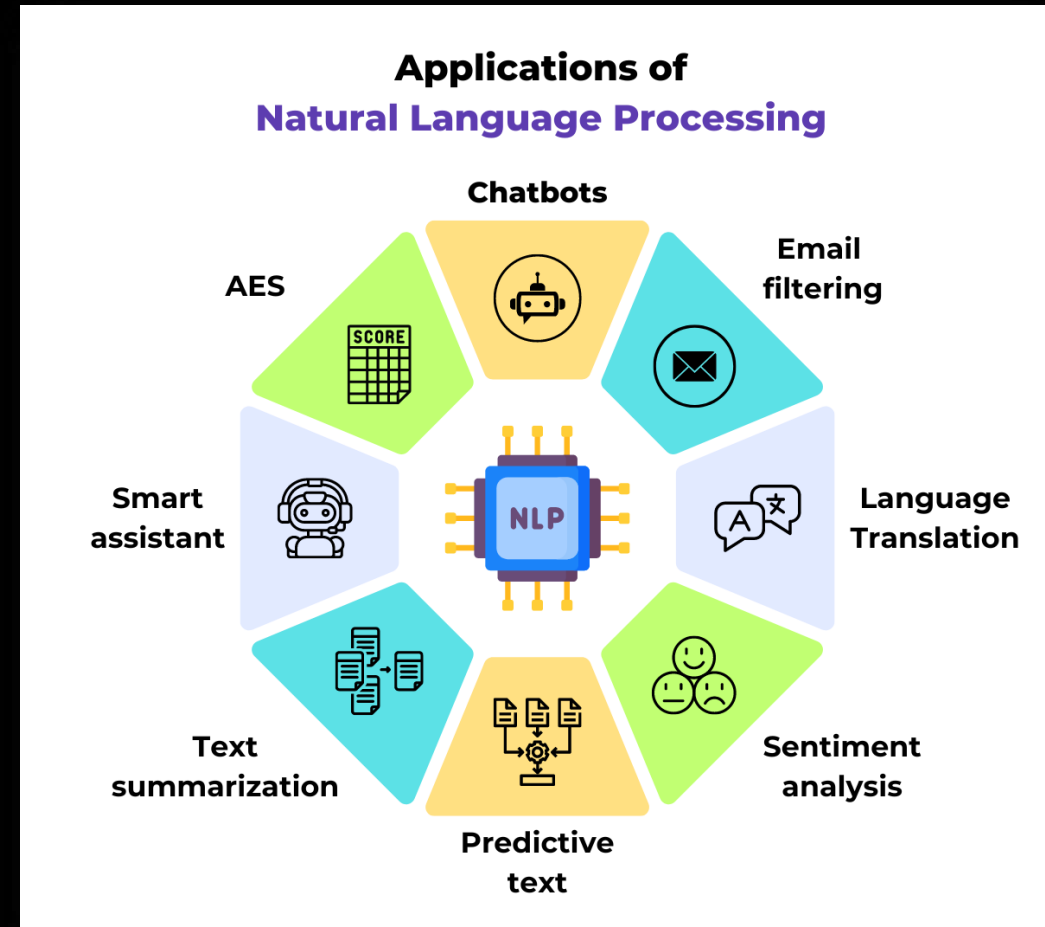
Text classification

Text summarization

Natural language  
generation (NLG)

# NLP Applications

- NLP makes it easier for humans to communicate and collaborate with machines, by allowing them to do so in the natural human language they use every day.
- This offers benefits across many industries and applications.
  - Automation of repetitive tasks
  - Improved data analysis and insights
  - Enhanced search
  - Content generation



# NLP Approaches

---

## NLP approaches

**Rules & Heuristics  
based NLP**

**Statistical NLP**

**Deep learning  
NLP**

# NLP Approaches

---

## Rules-based NLP

- The earliest NLP applications were simple if-then decision trees, requiring preprogrammed rules.
- They are only able to provide answers in response to specific prompts.
- Because there is no machine learning or AI capability in rules-based NLP, this function is highly limited and not scalable.

# NLP Approaches

---

## Statistical NLP

- Statistical NLP automatically extracts, classifies and labels elements of text and voice data and then assigns a statistical likelihood to each possible meaning of those elements. This relies on machine learning, enabling a sophisticated breakdown of linguistics such as part-of-speech tagging.

Statistical NLP introduced the essential technique of mapping language elements—such as words and grammatical rules—to a vector representation so that language can be modeled by using mathematical (statistical) methods, including regression or Markov models.

# NLP Approaches

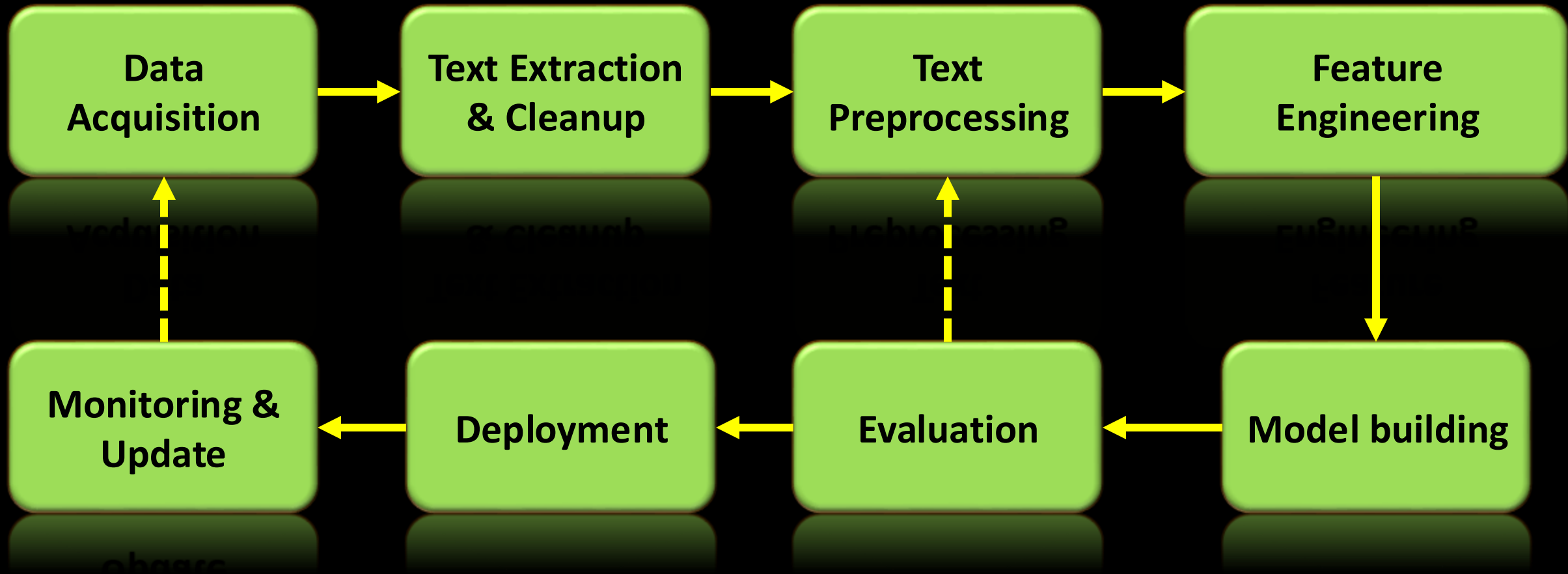
---

## Deep learning NLP

- Deep learning models have become the dominant mode of NLP, by using huge volumes of raw, unstructured data—both text and voice—to become ever more accurate.
- Deep learning can be viewed as a further evolution of statistical NLP, with the difference that it uses neural network models.



# ML&DL - based NLP Flowchart



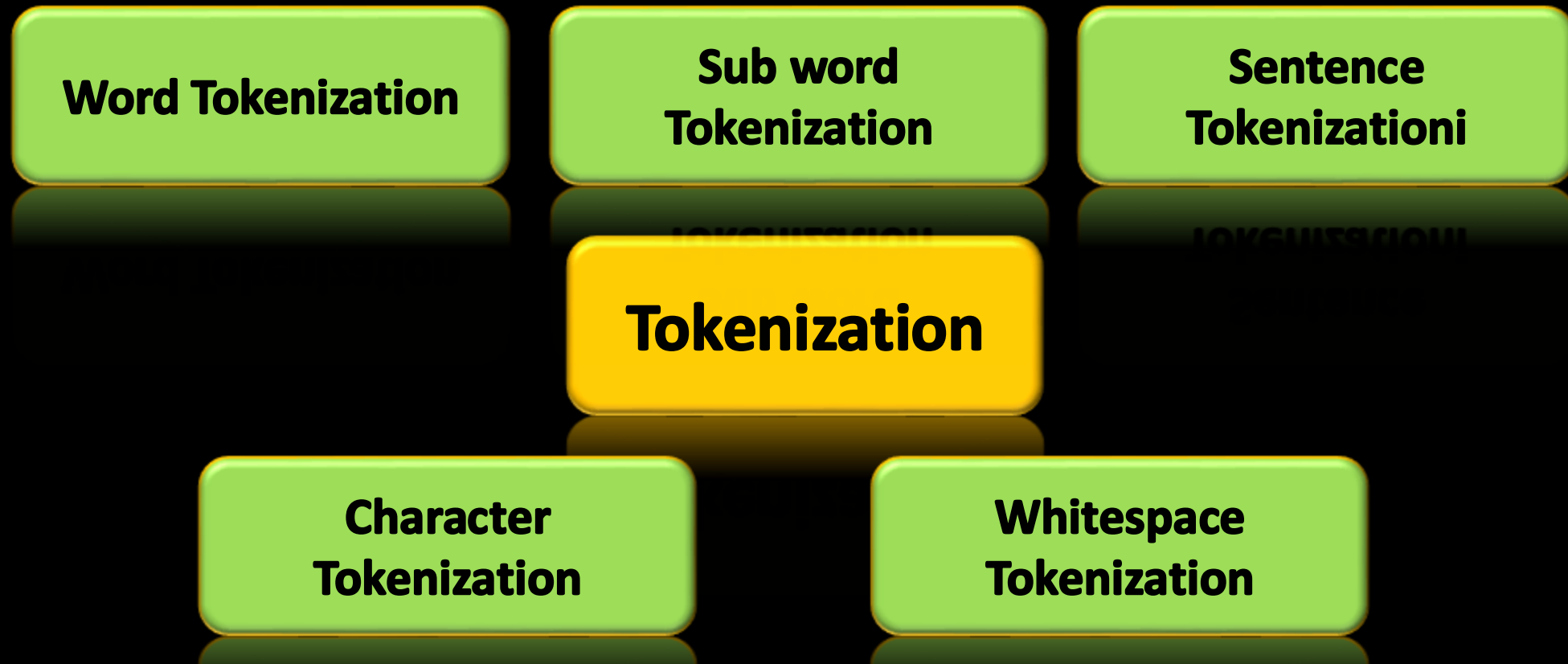
# Text Cleaning and Preprocessing

---

- Tokenization.
- Stop word removal.
- Stemming and lemmatization.

# Text Cleaning and Preprocessing

---



# Text Cleaning and Preprocessing

## Stop word removal

---

**Definition:** Stop words are common words in a language (e.g., “is”, “the”, “and”, “of”) that are often removed during text processing because they provide little meaningful information for many NLP tasks, such as classification or generalization. Removing stop words helps reduce noise and focus on the main content.

### Example:

**Input:** "This is an example of a sentence that has stop words."

**Output:** "example sentence stop words"

# Text Cleaning and Preprocessing

## Stemming

---

**Definition:** Stemming is the process of reducing a word to its root or basic form (stem) by cutting off suffixes, often without understanding its meaning. This is a rule-based, crude approach that can produce words that are not part of the dictionary.

### Example:

**Input:** "running", "runner", "ran"

**Output:** "run", "runner", "ran"

# Text Cleaning and Preprocessing

## Lemmatization

---

**Definition:** Lemmatization reduces words to their base or dictionary form (lemma), taking into account the meaning and part of speech of the word. It ensures that the resulting word is correct and meaningful.

### Example:

**Input:** "running", "better", "feet"

**Output:** "run", "good", "foot"

# Text Cleaning and Preprocessing

## Comparison

| Feature            | Stemming                                  | Lemmatization                         |
|--------------------|-------------------------------------------|---------------------------------------|
| Approach           | Rule-based (cuts suffixes/prefixes).      | Dictionary-based (meaning-aware).     |
| Output             | May not be a valid word (e.g., "easili"). | Always a valid word (e.g., "easily"). |
| Speed              | Faster due to simple rules.               | Slower due to more complexity.        |
| Example: "running" | "run"                                     | "run"                                 |
| Example: "better"  | "better"                                  | "good"                                |

# Basic Encoding Techniques

---

## One-Hot Encoding

- Represents each word as a binary vector of length equal to the vocabulary size.
- Each vector has one 1 at the index corresponding to the word and 0 elsewhere.
- **Pros:** Simple and easy to implement.
- **Cons:** Results in very sparse and high-dimensional vectors.



# Basic Encoding Techniques

---

## One-Hot Encoding (Example)

**Vocabulary:** ["cat", "dog", "fish", "bird"]

### Step 1: Assign an index to each word

- cat  $\rightarrow$  0
- dog  $\rightarrow$  1
- fish  $\rightarrow$  2
- bird  $\rightarrow$  3

### Step 2: Create one-hot vectors

Each word is represented as a binary vector of length equal to the vocabulary size (4 in this case).

Only the index corresponding to the word is 1, and all other indices are 0.

- cat  $\rightarrow$  [1, 0, 0, 0]
- dog  $\rightarrow$  [0, 1, 0, 0]
- fish  $\rightarrow$  [0, 0, 1, 0]
- bird  $\rightarrow$  [0, 0, 0, 1]

# Basic Encoding Techniques

---

## Bag of Words (BoW)

- Represents text as a vector of word counts or frequencies.
- Ignores word order, treating text as a "bag" of words.
- **Pros:** Simple and effective for smaller datasets.
- **Cons:** Loses information about word order and context.

# Basic Encoding Techniques

---

## Bag of Words (Example)

**Sentences1:** “Hi, I love NLP”

**Sentences2:** “I am learning NLP now”

### Step 1: Build the Vocabulary

- Create a unique list of all words in the dataset (sentences).
- **Vocabulary:** [“Hi”, “I”, “love”, “NLP”, “am”, “learning”, “now”]

### Step 2: Assign an Index to Each Word

- Hi  $\rightarrow$  0
- I  $\rightarrow$  1
- love  $\rightarrow$  2
- NLP  $\rightarrow$  3
- am  $\rightarrow$  4
- learning  $\rightarrow$  5
- now  $\rightarrow$  6

# Basic Encoding Techniques

---

## Step 3: Represent Each Sentence as a Vector

For each sentence, count the occurrences of each word from the vocabulary.

**Sentence 1:** " Hi, I love NLP "

Vector: [1, 1, 1, 1, 0, 0, 0]

**Sentence 2:** " I am learning NLP now "

Vector: [0, 1, 0, 1, 1, 1, 1]

## Step 2: Assign an Index to Each Word

- Hi  $\rightarrow$  0
- I  $\rightarrow$  1
- love  $\rightarrow$  2
- NLP  $\rightarrow$  3
- am  $\rightarrow$  4
- learning  $\rightarrow$  5
- now  $\rightarrow$  6

# Basic Encoding Techniques

---

## TF-IDF (Term Frequency-Inverse Document Frequency)

- Extends BoW by weighting words based on their importance in a document relative to the entire corpus.
- Highlights rare but significant words.
- **Pros:** Reduces the impact of common but uninformative words.
- **Cons:** Still loses context and word order.

# Basic Encoding Techniques

---

## TF-IDF (Example)

**Document1:** “Hi, I love NLP”

**Document2:** “I am learning NLP now”

## Step 1: Build the Vocabulary

- Create a unique list of all words in the dataset (sentences).
- **Vocabulary:** [“Hi”, “I”, “love”, “NLP”, “am”, “learning”, “now”]

# Basic Encoding Techniques

---

## TF-IDF (Example)

### Step 2: Calculate Term Frequency (TF)

TF measures how often a word appears in a document. For simplicity, let's assume:

$$TF(t, d) = \frac{\text{Number of occurrences of term } t \text{ in document } d}{\text{Total terms in document } d}$$

# Basic Encoding Techniques

## TF-IDF (Example)

### Step 2: Calculate Term Frequency (TF)

TF measures how often a word appears in a document. For simplicity, let's assume:

| Words    | TF in Document 1 ("Hi, I love NLP") | TF in Document 2 ("I am learning NLP now") |
|----------|-------------------------------------|--------------------------------------------|
| Hi       | $\frac{1}{4}=0.25$                  | $\frac{0}{5}=0.0$                          |
| I        | $\frac{1}{4}=0.25$                  | $\frac{1}{5}=0.2$                          |
| love     | $\frac{1}{4}=0.25$                  | $\frac{0}{5}=0.0$                          |
| NLP      | $\frac{1}{4}=0.25$                  | $\frac{1}{5}=0.2$                          |
| am       | $\frac{0}{4}=0.0$                   | $\frac{1}{5}=0.2$                          |
| learning | $\frac{0}{4}=0.0$                   | $\frac{1}{5}=0.2$                          |
| now      | $\frac{0}{4}=0.0$                   | $\frac{1}{5}=0.2$                          |



# Basic Encoding Techniques

---

## TF-IDF (Example)

### Step 3: Calculate Inverse Document Frequency (IDF)

IDF reduces the weight of common words that appear in many documents. It is calculated as:

$$IDF(t) = \ln \left( \frac{\text{Total number of documents}}{\text{Number of documents containing } t} \right)$$

# Basic Encoding Techniques

## TF-IDF (Example)

### Step 3: Calculate Inverse Document Frequency (IDF)

| Words    | Documents Containing Word | IDF Value                            |
|----------|---------------------------|--------------------------------------|
| Hi       | 1                         | $\ln\left(\frac{2}{1}\right) = 0.69$ |
| I        | 2                         | $\ln\left(\frac{2}{2}\right) = 0.0$  |
| love     | 1                         | $\ln\left(\frac{2}{1}\right) = 0.69$ |
| NLP      | 2                         | $\ln\left(\frac{2}{2}\right) = 0.0$  |
| am       | 1                         | $\ln\left(\frac{2}{1}\right) = 0.69$ |
| learning | 1                         | $\ln\left(\frac{2}{1}\right) = 0.69$ |
| now      | 1                         | $\ln\left(\frac{2}{1}\right) = 0.69$ |

# Basic Encoding Techniques

## TF-IDF (Example)

### Step 4: Calculate TF-IDF

Multiply the TF values by the corresponding IDF values.

**Document1:** “Hi, I love NLP”

$$TF - IDF = [0.25 \cdot 0.69, 0.25 \cdot 0, 0.25 \cdot 0.69, 0.25 \cdot 0, 0 \cdot 0.69, 0 \cdot 0.69, 0 \cdot 0.69]$$

$$TF - IDF = [0.1725, 0, 0.1725, 0, 0, 0, 0]$$

**Document2:** “I am learning NLP now”

$$TF - IDF = [0 \cdot 0.69, 0.2 \cdot 0, 0 \cdot 0.69, 0.2 \cdot 0, 0.2 \cdot 0.69, 0.2 \cdot 0.69, 0.2 \cdot 0.69]$$

$$TF - IDF = [0, 0, 0, 0, 0.138, 0.138, 0.138]$$

# Distributed Word Representations

---

These techniques capture semantic meaning and context.

## Word Embeddings (e.g., Word2Vec, GloVe, FastText)

- Represent words as dense, fixed-length vectors learned from large corpora.
- Words with similar meanings have similar vector representations.
- **Pros:** Captures semantic similarity and can be pre-trained.
- **Cons:** Static embeddings don't account for polysemy.

# Distributed Word Representations

```
from gensim.models import KeyedVectors

# Step 1: Load the pre-trained Word2Vec model (Google News 300)
# Note: This is a large file (~1.5GB). Ensure sufficient memory is available.
!wget -q https://s3.amazonaws.com/dl4j-distribution/GoogleNews-vectors-negative300.bin.gz
word2vec_model = KeyedVectors.load_word2vec_format("GoogleNews-vectors-negative300.bin.gz", binary=True)

# Step 2: Words to convert
words = ["King", "Man", "Woman", "Queen"]

# Step 3: Convert words to vectors
for word in words:
    if word in word2vec_model:
        vector = word2vec_model[word]
        print(f"Vector for '{word}':\n{vector[:10]}...") # Display first 10 elements of the vector
        print(f"Vector size: {len(vector)}\n")
    else:
        print(f"'{word}' not found in the Word2Vec model.")
```



# Distributed Word Representations

Vector for 'King':

```
[0.21679688 0.12597656 -0.12597656 0.12597656 -0.15234375 0.10058594  
0.23632812 -0.0859375 -0.16601562 0.01757812]...
```

Vector size: 300

Vector for 'Man':

```
[-0.00585938 0.0078125 -0.00976562 0.06445312 -0.05273438 0.02734375  
0.03417969 -0.02148438 -0.04003906 0.07226562]...
```

Vector size: 300

Vector for 'Woman':

```
[-0.02539062 0.05078125 -0.02050781 0.01757812 -0.03222656 -0.03808594  
0.08203125 -0.0546875 -0.02636719 0.04296875]...
```

Vector size: 300

Vector for 'Queen':

```
[0.05859375 0.10449219 -0.11816406 0.12597656 -0.03222656 -0.07617188  
0.00878906 -0.04785156 -0.08203125 0.06347656]...
```

Vector size: 300

# Distributed Word Representations

- **Example: "King - Man + Woman = Queen"**
- **Step 1: Vectors for Each Word**

Using the Word2Vec model, you retrieve the dense vectors for the words "King", "Man", "Woman", and "Queen". Each vector is a 300-dimensional numerical representation.
- **Step 2: Arithmetic on Vectors**

Word embeddings encode semantic information. For instance:  
"King" represents "a male monarch."  
"Man" represents "male."  
"Woman" represents "female."
- **When you compute:**  
King - Man + Woman,  
it effectively removes the "male" concept from "King" and adds the "female" concept, resulting in a vector similar to "Queen".

# Contextualized Word Representations

---

These are dynamic and depend on the context in which a word appears.

## Transformer-based Models (e.g., BERT, GPT, RoBERTa, T5)

- Use attention mechanisms to generate contextualized embeddings for each word in a sentence.
- **Pros:** State-of-the-art performance on many NLP tasks.
- **Cons:** High computational cost and requires significant training resources.



# Contextualized Word Representations

```
from transformers import BertTokenizer, BertModel
import torch

# Step 1: Load BERT model and tokenizer
model_name = "bert-base-uncased" # Pretrained BERT model
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertModel.from_pretrained(model_name)

# Step 2: Tokenize the input sentence
sentence = "I am learning Natural Language Processing now"
inputs = tokenizer(sentence, return_tensors="pt", add_special_tokens=True)

# Step 3: Get embeddings from BERT
with torch.no_grad():
    outputs = model(**inputs)
    # Last hidden state has embeddings for each token in the sentence
    hidden_states = outputs.last_hidden_state

# Step 4: Map tokens to embeddings
tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"].squeeze())
embeddings = hidden_states.squeeze()

# Display token embeddings
for token, embedding in zip(tokens, embeddings):
    print(f"Token: {token}")
    print(f"Embedding: {embedding[:5]}..." ) # Show first 5 values for brevity
    print()
```

# Contextualized Word Representations

```
Token: [CLS]
Embedding: tensor([ 0.1987,  0.2543, -0.0283, -0.3329, -0.1896])...

Token: i
Embedding: tensor([ 0.5118,  0.1275, -0.0119, -0.3459, -0.0161])...

Token: am
Embedding: tensor([ 0.1765,  0.6725, -0.1484, -0.7117,  0.6979])...

Token: learning
Embedding: tensor([-0.5310,  0.3913,  0.2965, -0.1120,  0.0529])...

Token: natural
Embedding: tensor([ 0.2229,  0.2083, -0.1043, -0.6550,  0.3019])...

Token: language
Embedding: tensor([-0.4395,  0.2574,  0.4251, -0.9594,  0.1016])...

Token: processing
Embedding: tensor([ 0.4494, -0.1933, -0.2059, -0.4707,  0.1882])...

Token: now
Embedding: tensor([-0.6280, -0.2269, -0.1857, -0.2869,  0.1550])...

Token: [SEP]
Embedding: tensor([ 0.7580,  0.2497, -0.2497,  0.5008, -0.2283])...
```



